



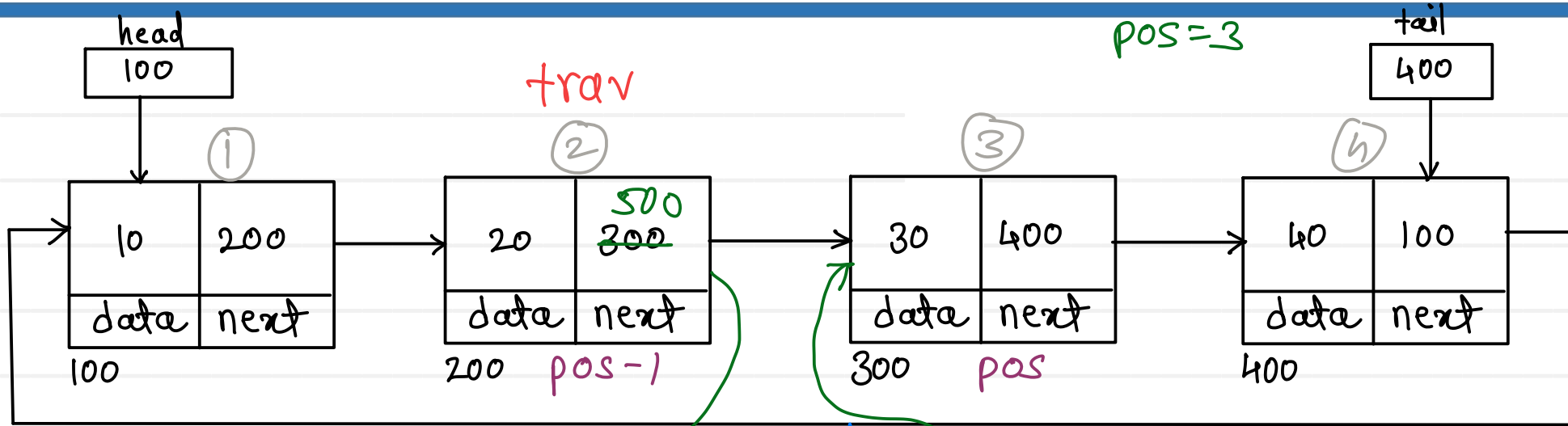
Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

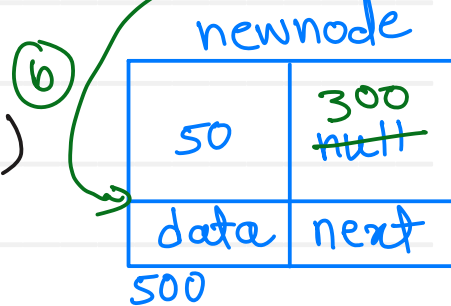
Singly Circular Linked List - Add position



Node trav = head;

for(i = 1 ; i < pos - 1 ; i++)

trav = trav.next;



(a) if list is not empty,
// traverse till pos-1 node
a. add pos node into next of newnode
b. add newnode into next pos-1 node

pos = 3
trav i i < 2
100 1 T
200 2 F

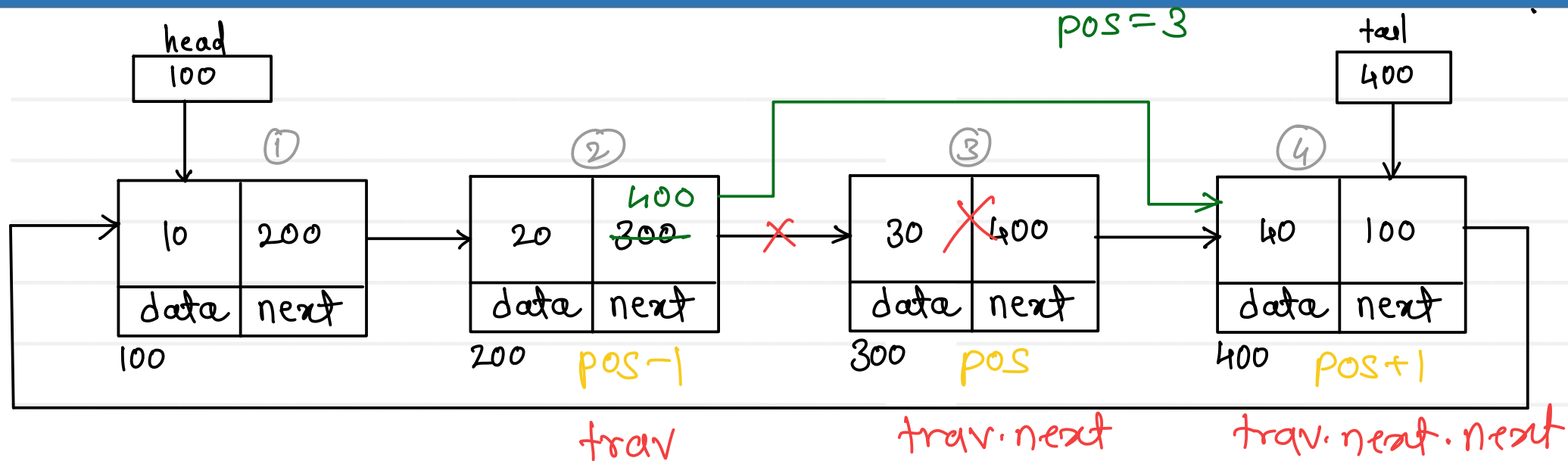
pos = 5
trav i i < 4
100 1 T
200 2 T
300 3 T
400 4 F

pos = 1
trav i i < 0
100 1 F

pos = -1
trav i i < -2
100 1 F

$$T(n) = O(n)$$

Singly Circular Linked List - Delete position

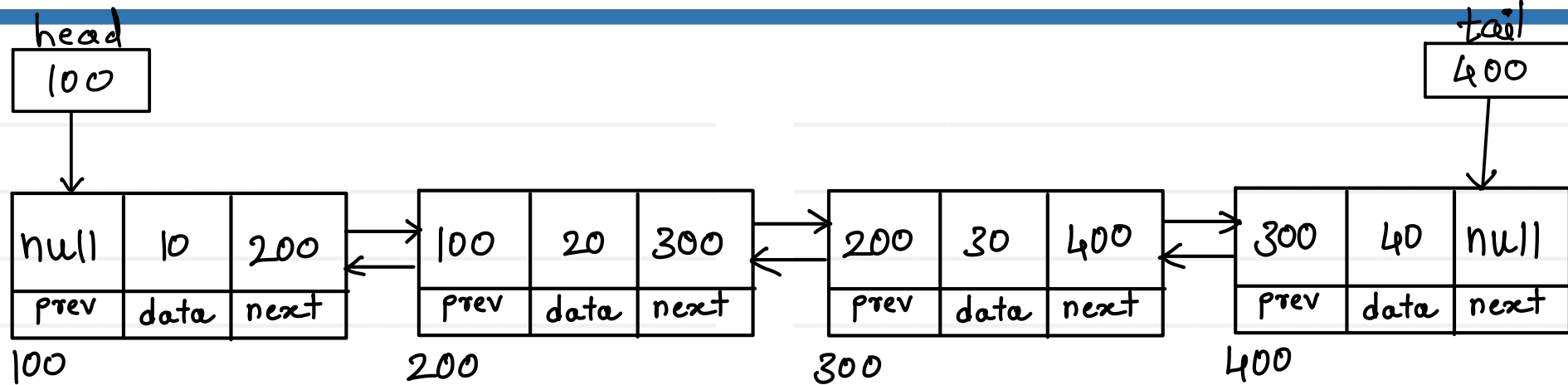


if list is not empty,

- traverse till **pos-1** node
- add **pos+1** node into next **pos-1** node

$$T(n) = O(n)$$

Doubly Linear Linked List - Display

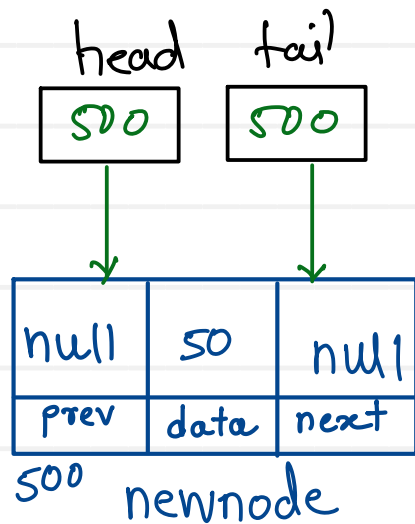
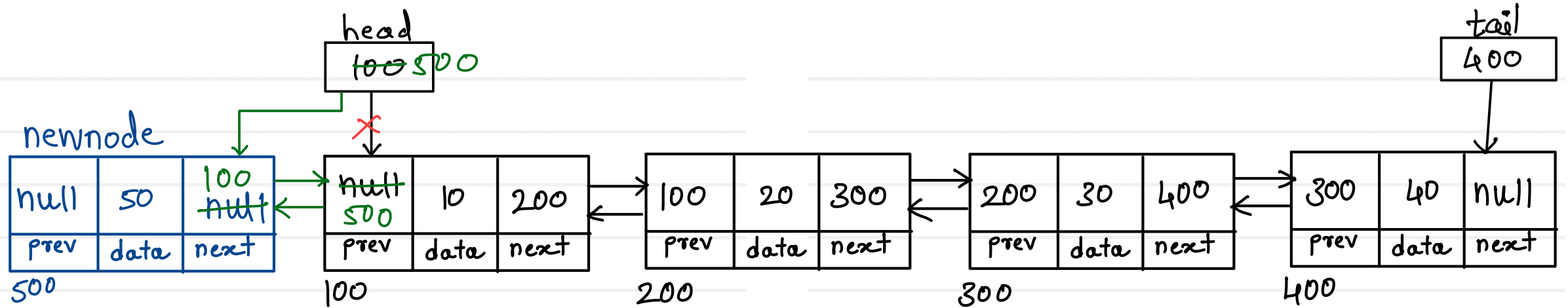


1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

$$T(n) = O(n)$$

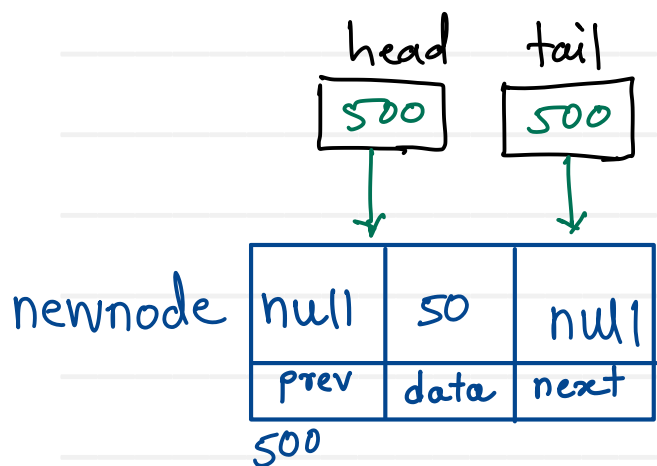
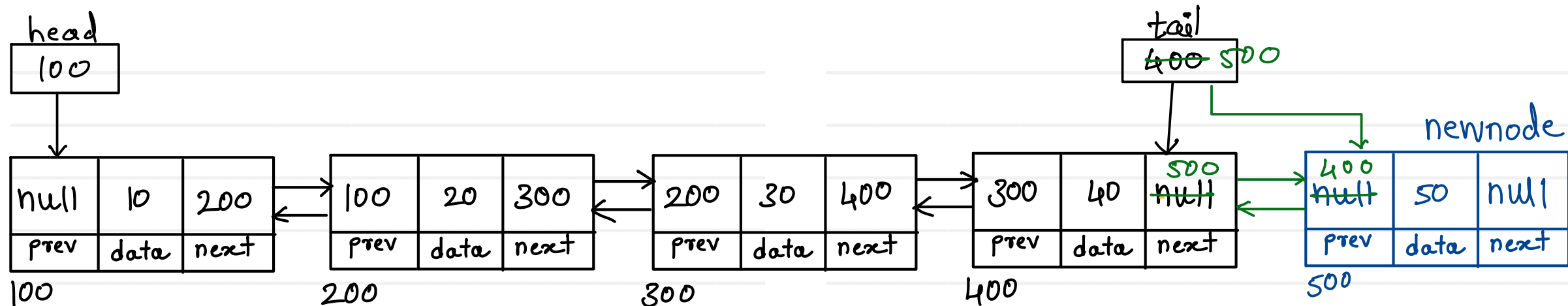
Doubly Linear Linked List - Add first



1. create newnode
2. if list is empty, add newnode into head & tail
3. if list is not empty,
 - a. add first node into next of newnode
 - b. add newnode into prev of first node
 - c. move head on newnode

$$T(n) = O(1)$$

Doubly Linear Linked List - Add last

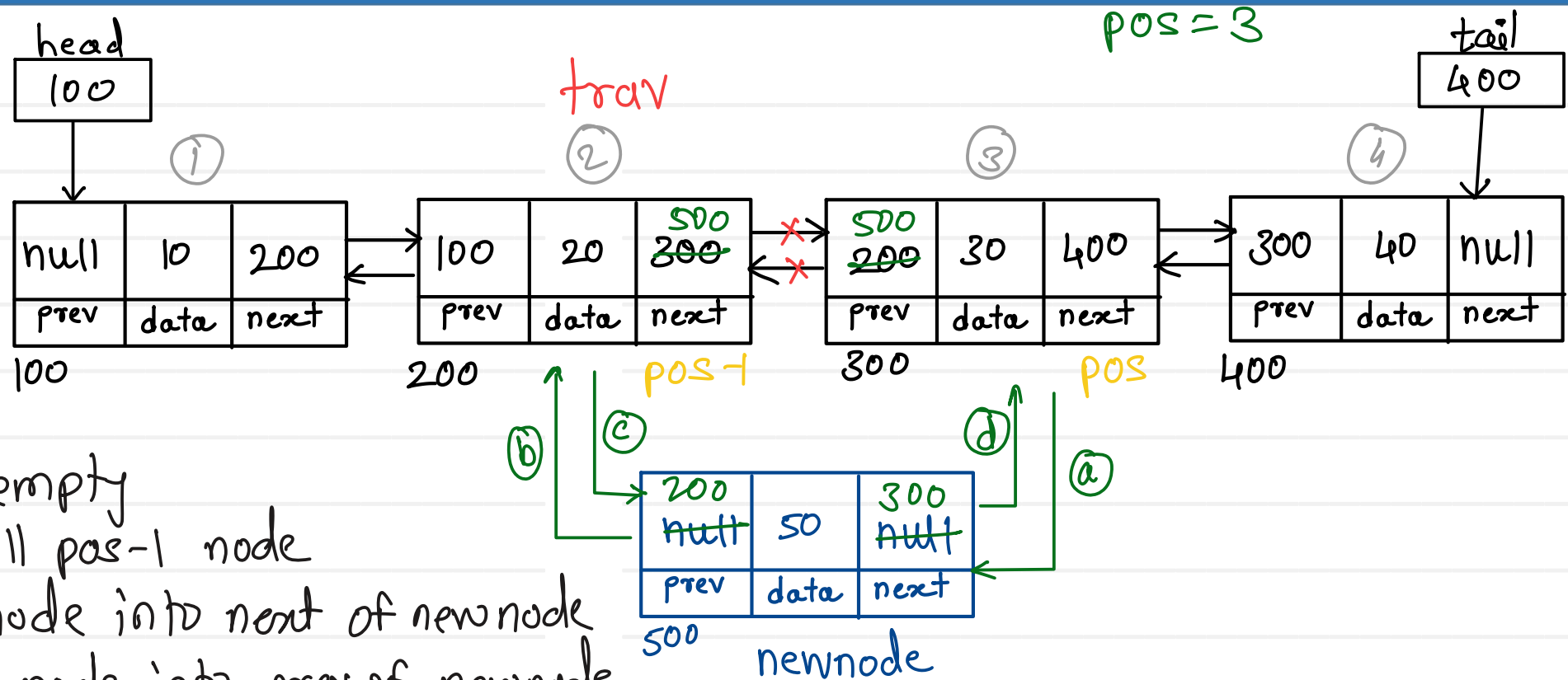


if list is empty,
then add newnode
into head & tail

if list is not empty,
a. add last node into prev of newnode
b. add newnode into next of last node
c. move tail on newnode

$$T(n) = O(1)$$

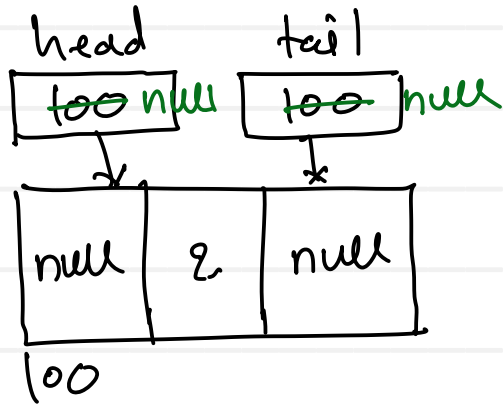
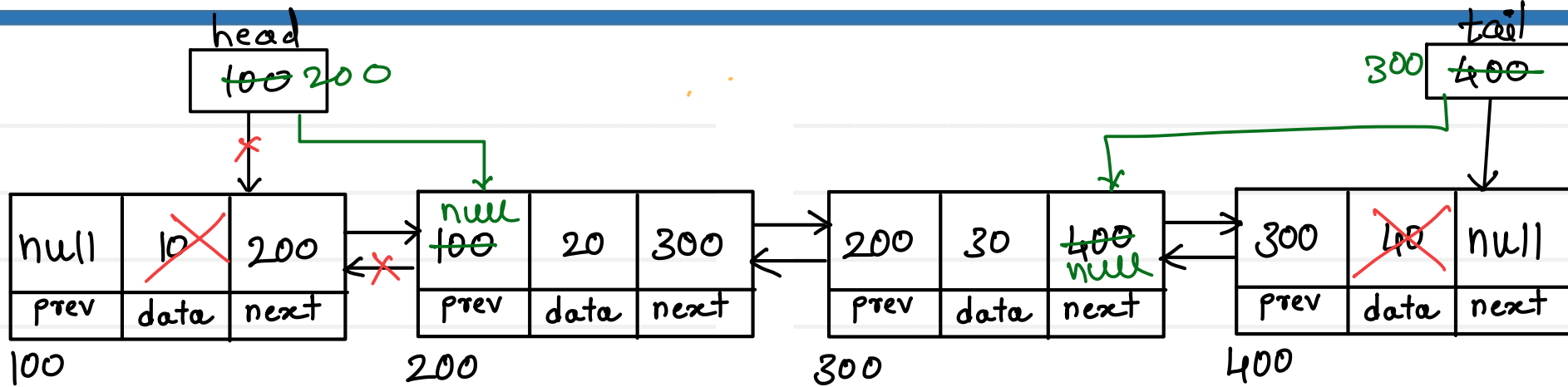
Doubly Linear Linked List - Add position



if list is not empty
 - traverse till pos-1 node
 a. add pos node into next of newnode
 b. add pos-1 node into prev of newnode
 c. add newnode into next of pos-1 node
 d. add newnode into prev of pos node

$$T(n) = O(n)$$

Doubly Linear Linked List - Delete first

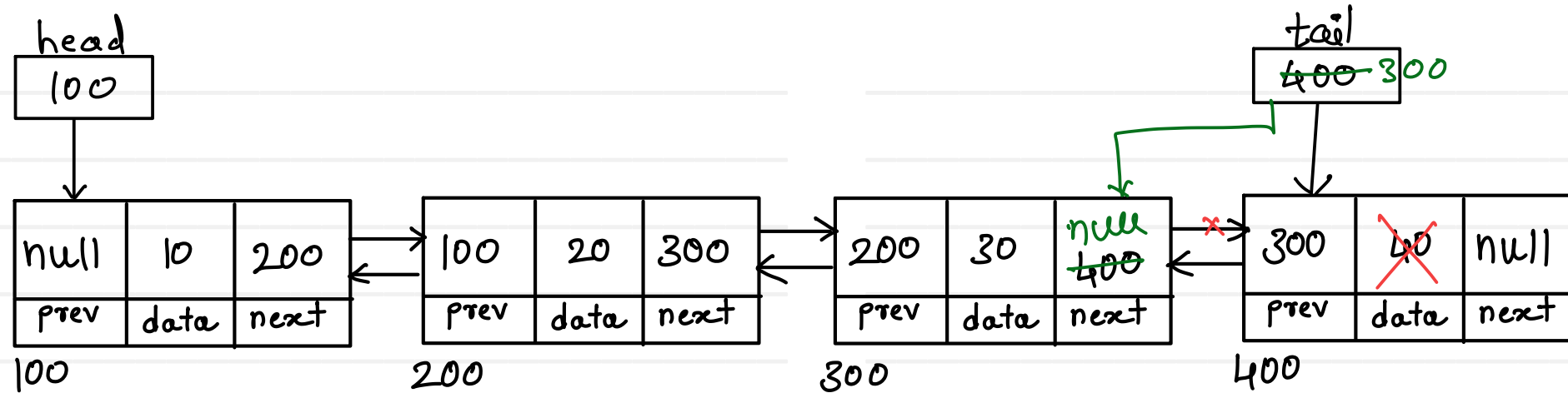


head.next.prev = null;
head = head.next;

$$T(n) = O(1)$$

1. if list is empty, return
2. if list has single node,
head = tail = null
3. if list has multiple nodes
 - a. add null into prev of second node
 - b. move head on second node

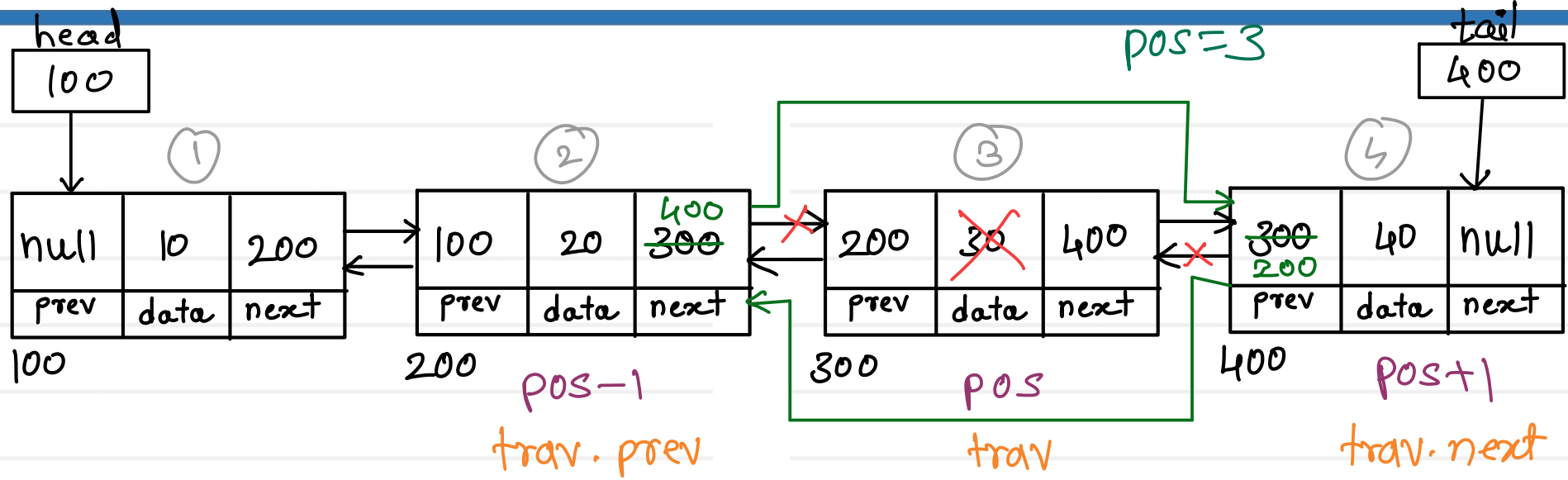
Doubly Linear Linked List - Delete last



1. if list is empty, return
2. if list has single node, delete it
head = tail = null
3. if list has multiple nodes,
 - a. move tail on second last node
 - b. add null into next of second last node

$$T(n) = O(1)$$

Doubly Linear Linked List - Delete Position

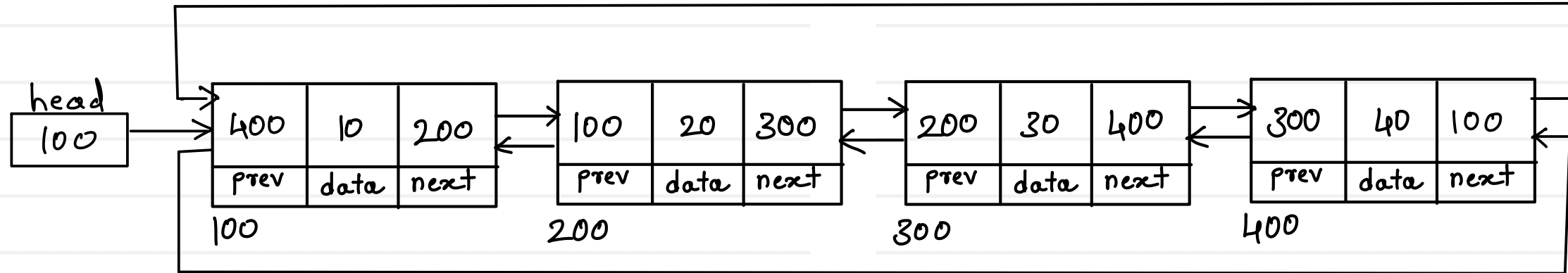


1. traverse till pos node
2. add pos+1 node into next of pos-1 node
3. add pos-1 node into prev of pos+1 node

$trav.prev.next = trav.next$
 $trav.next.prev = trav.prev$

$$T(n) = O(n)$$

Doubly Circular Linked List - Display

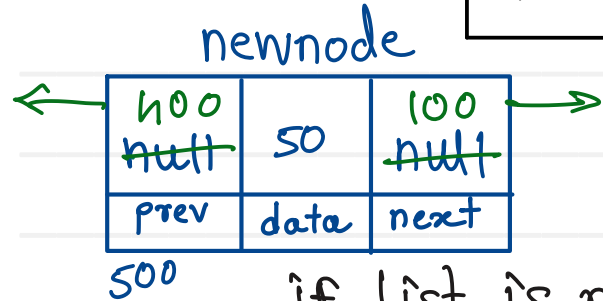
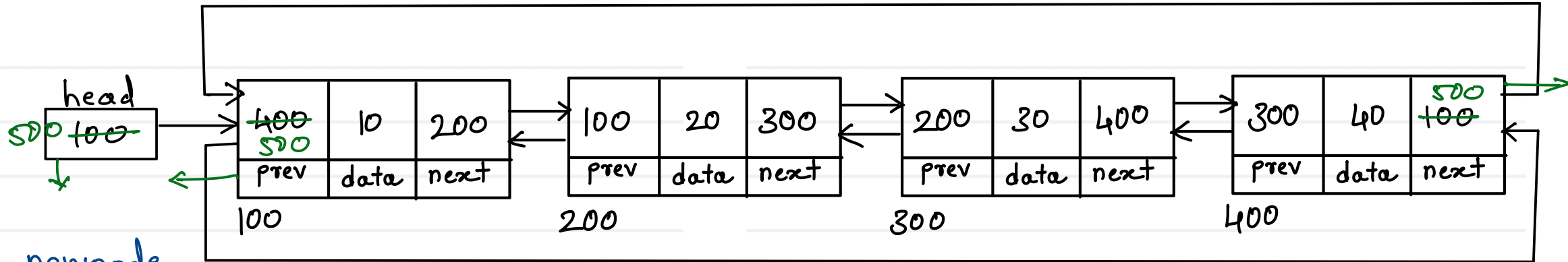


1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

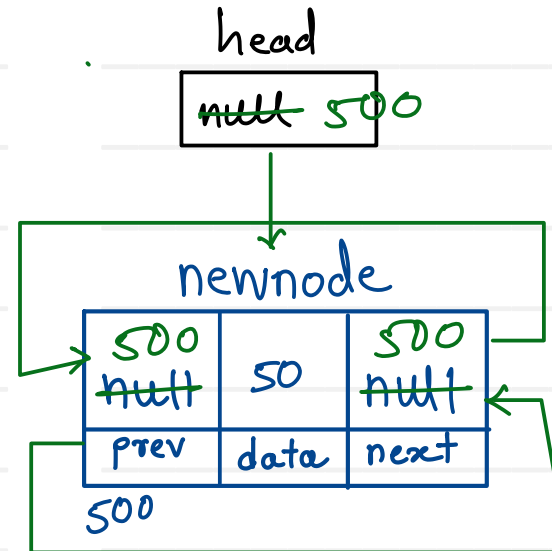
1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

$$T(n) = O(n)$$

Doubly Circular Linked List - Add first



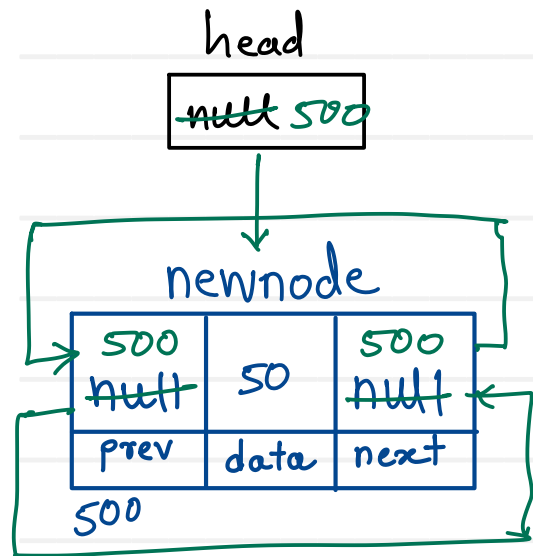
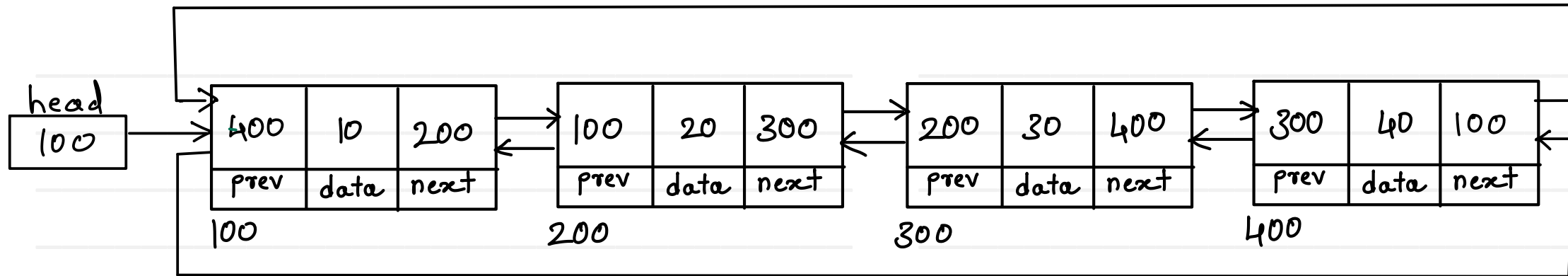
- if list is not empty
- newnode.next = first node
 - newnode.prev = last node
 - last node next = newnode
 - first node prev = newnode
 - head = newnode



- if list is empty
- add newnode into head
 - make list circular

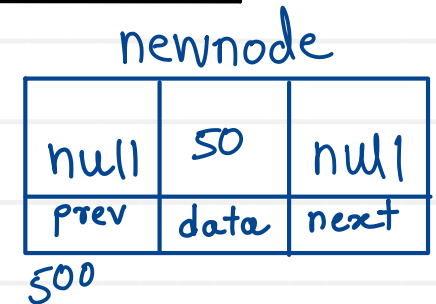
$$T(n) = O(1)$$

Doubly Circular Linked List - Add last

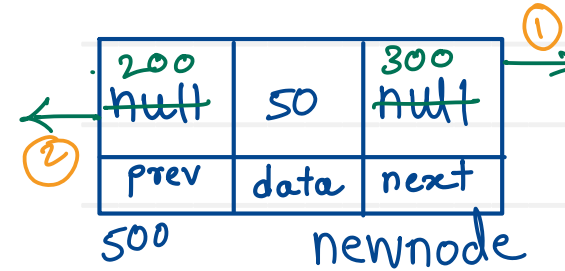


if list empty,
a. add newnode into head
b. make list circular

$$T(n) = O(1)$$

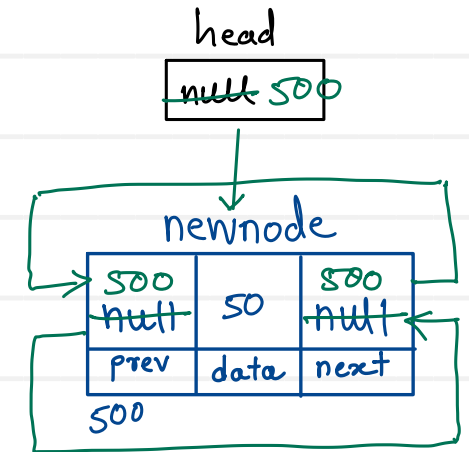


if list is not empty,
a. next of newnode = first node
b. prev of newnode = last node
c. next of last node = newnode
d. prev of first node = newnode

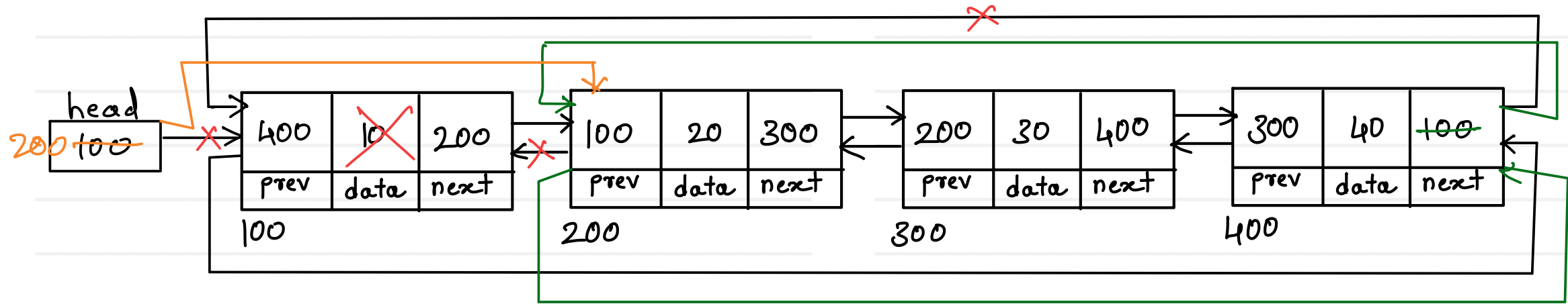


1. create node
 2. if list is empty
 - a. add newnode into head
 - b. make list circular
 3. if list is not empty.
 - a. traverse till pos-1 node
 - b. add pos node into next of newnode
 - c. add pos-1 node into prev of newnode
 - d. add newnode into next of pos-1 node
 - e. add newnode into prev of pos node
- 

$$T(n) = O(1)$$



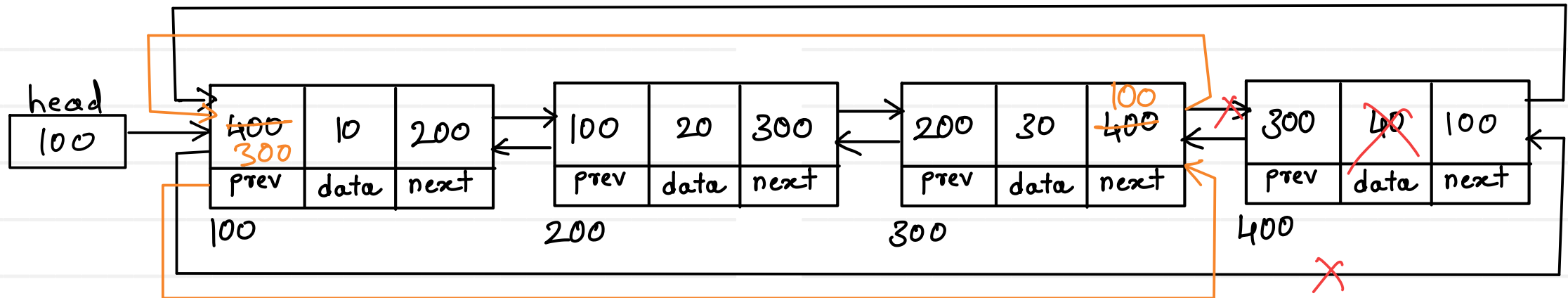
Doubly Circular Linked List - Delete first



1. if list is empty, return
2. if list has single node, head = null
3. if list has multiple nodes,
 - a. add second node into next of last node
 - b. add last node into prev of second node
 - c. move head on second node

$$T(n) = O(1)$$

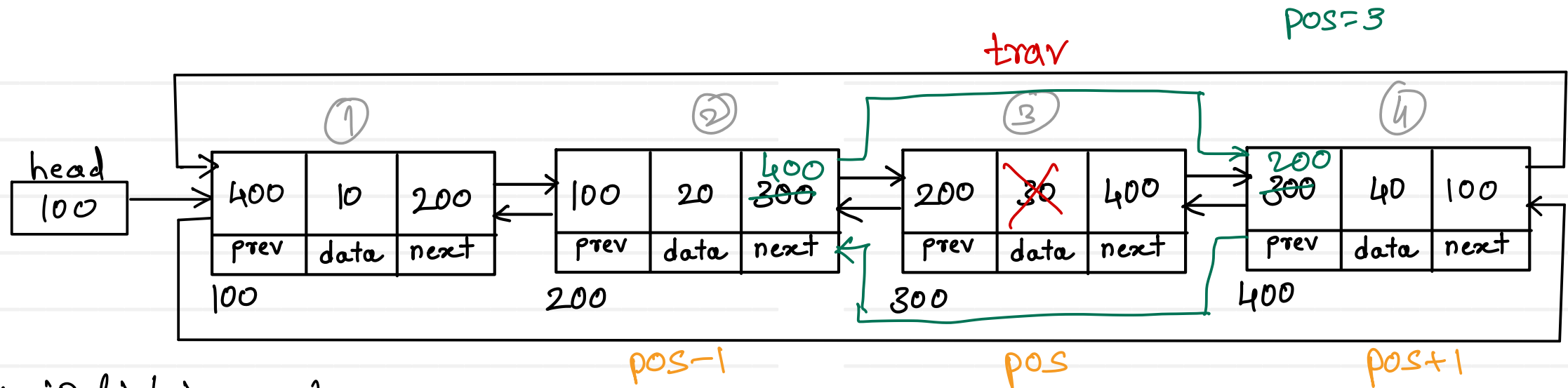
Doubly Circular Linked List - Delete last



1. if list is empty, return
2. if list has single node, head = null
3. if list has multiple nodes,
 - a. add second last node into prev of first node
 - b. add first node into next of second last node

$$T(n) = O(1)$$

Doubly Circular Linked List - Delete position



1. if list is empty
return;
2. if list has single node
head = tail = null;
3. if list has multiple nodes
 - a. traverse till pos node
 - b. add pos+1 node into next of pos-1 node
 - c. add pos-1 node into prev of pos+1 node

trav → pos
trav.prev → pos-1
trav.next → pos+1

$$T(n) = O(n)$$

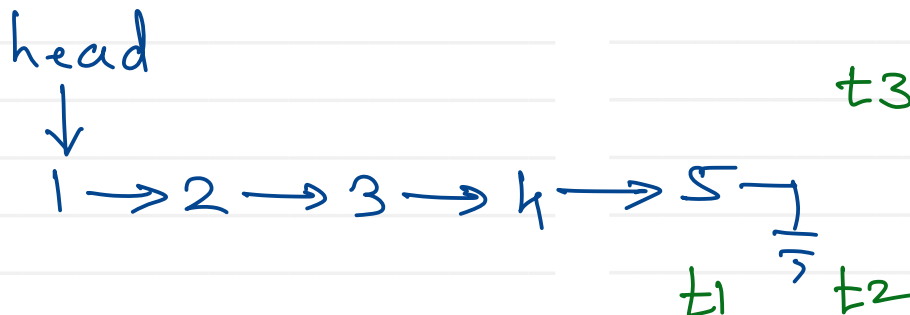
Reverse Linked List

Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:

Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]



Example 2:

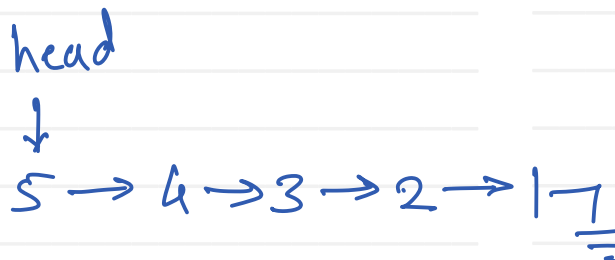
Input: head = [1,2]

Output: [2,1]

Example 3:

Input: head = []

Output: []



$$T(n) = O(n)$$

$$S(n) = O(1)$$

```
mid reverseList( ) {
    t1 = null;
    t2 = head;
    while (t2 != null) {
        t3 = t2.next;
        t2.next = t1;
        t1 = t2;
        t2 = t3;
    }
    head = t1;
}
```

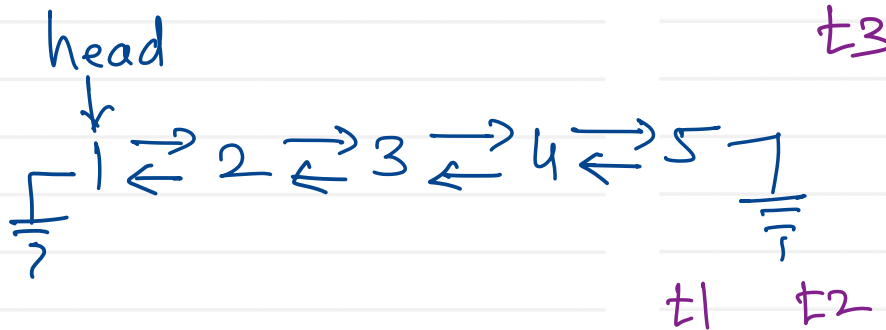
Reverse Linked List

Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:

Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]



Example 2:

Input: head = [1,2]

Output: [2,1]

Example 3:

Input: head = []

Output: []



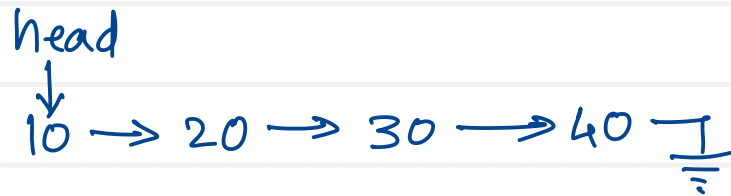
$$T(n) = O(n)$$

$$S(n) = O(1)$$

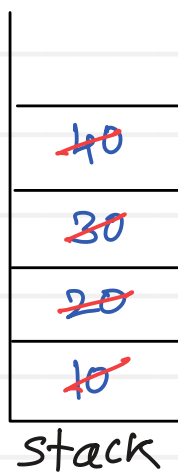
```
Node reverseList ( head ) {
    Node t1 = head;
    Node t2 = head.next;
    Node t3;
    t1.next = null;
    while ( t2 != null ) {
        t3 = t2.next;
        t2.next = t1;
        t1.prev = t2;
        t1 = t2;
        t2 = t3;
    }
    head = t1;
    return head;
}
```

Singly linear linked list - Display reverse

1. push all node values on stack
2. pop all values from stack & print them one by one



Output : 40 30 20 10



```

void displayReverse( ) {
    Stack<Integer> st = new Stack<>();
    Node trav = head;
    while (trav != null) {           ← n times
        st.push(trav.data);
        trav = trav.next;
    }
    while ( !st.isEmpty() )         ← n times
        sysout( st.pop() );
}

```

$2n$

stack size = n

space $\propto n$

$S(n) = O(n)$

time $\propto 2n$

$T(n) = O(n)$

- function calling itself is called as recursion
- every recursive call is done on reduced input.
- to use recursion, we should know,
 - formula / process in terms of itself
 - terminating / base condition
- for every recursive call stack frame is created & when call completes it is automatically destroyed.
- for recursion space requirement is more.

$$n! = n * (n-1)! \quad 1! = 0! = 1$$

$$b^i = b * b^{(i-1)} \quad b^1 = b \text{ or } b^0 = 1$$

$$F_n = F_{(n-1)} + F_{(n-2)} \quad T_0 = 0, T_1 = 1$$

2	9
	4
	2
	1

1
0
0
1

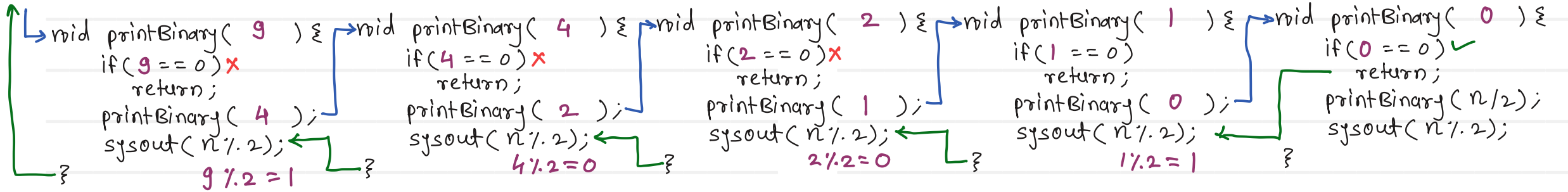
↑

$(9)_{10} = (1001)_2$

```

void printBinary(int n) {
    if(n == 0)
        return;
    printBinary(n/2);
    sysout(n%.2);
}
  
```

main()



Recursion

Direct

- function is directly called within itself

e.g. func() {

func()

}

Indirect (Mutual)

- multiple function call each other in circular way

e.g. fun1() { fun2() {

fun2() fun1()

}

}

tail

- recursive call will be last step in recursive function

head (non-tail)

- after completion of recursive call, some processing will be done

tree

- multiple recursive calls are done in a function

Iterative approach

– loops are used to implement algorithms

Time $\propto$ No. of iterations
of the loop

```
int factorial (int n) {  
    int fact = 1;  
    for (i = 1; i <= n; i++)  
        fact *= i;  
    return fact;  
}
```

No. of iterations = n | Extra space = i, fact

Time $\propto n$

$T(n) = O(n)$

$S(n) = O(1)$

Recursive approach

– recursion is used to implement algorithms

Time $\propto$ No. of recursive
calls of a function

```
5!  
4!  
3!  
2!  
1!  
int factorial (int n) {  
    if (n == 1)  
        return 1;  
    return n * factorial (n-1);  
}
```

No. of recursive calls = n

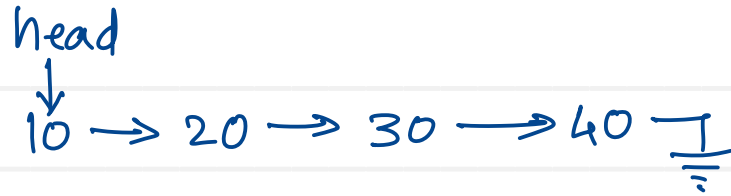
Time $\propto n$

$T(n) = O(n)$

n -FARs will be created
on stack

$S(n) = O(n)$

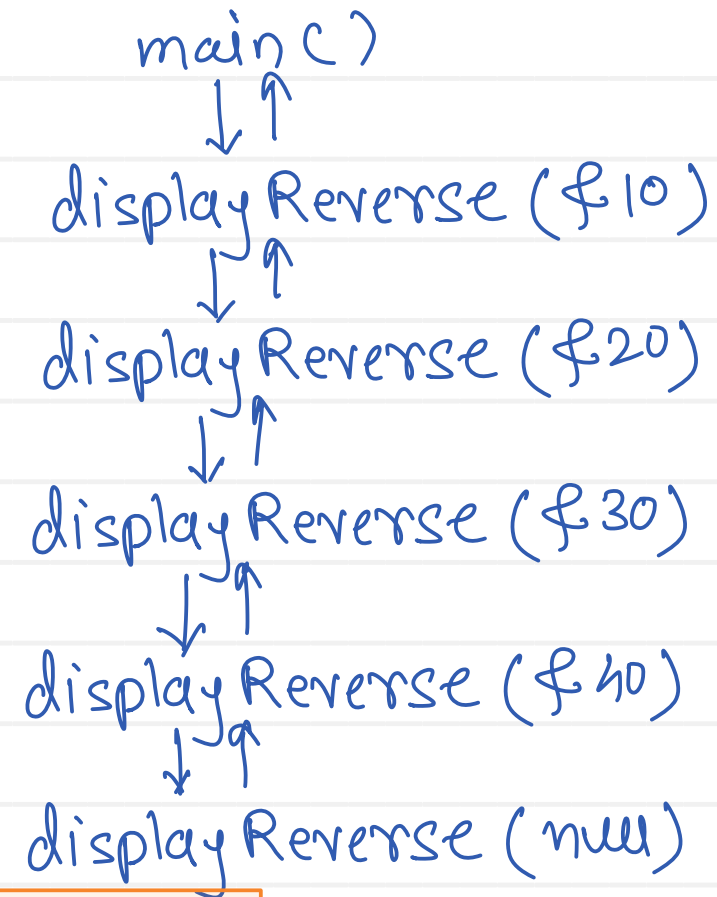
Singly linear linked list - Display reverse



```
void displayReverse (Node trav) {  
    if (trav == null)  
        return ;  
    displayReverse (trav.next);  
    sysout (trav.data);  
}
```

~

Output : 40 30 20 10



$$T(n) = O(n)$$
$$S(n) = O(n)$$

```
void reverseDisplay(Node trav) {  
    if (trav == null)  
        return;  
    reverseDisplay(trav.next);  
    sysout(trav.data);  
}
```

↑
non tail(head)
recursion

```
void forwardDisplay(Node trav) {  
    if (trav == null)  
        return;  
    sysout(trav.data);  
    forwardDisplay(trav.next);  
}
```

↑
tail recursion